

UNIT INFORMATION	
UNIT CODE	H7DT 04/VE/02
UNIT NAME	Web Application Development Project
ASSESSMENT TYPE	WEB APPLICATION DEVELOPMENT PROJECT, RECORDED SOLUTION DEMONSTRATION, AND LIVE VIVA VOCE EXAMINATION WITH SCENARIO-BASED CODING CHALLENGE
PROJECT TITLE	Web Application Development Project For CampusHub

STUDENT INFORMATION	
STUDENT NAME	M A Banuja Kaushalya
STUDENT ID	2024/2025/MA/SE/I112/059
BRANCH	Mathara
BATCH	Cheetah

1. Introduction

1.1 Background Scenario

CampusHub is a growing student services organisation that coordinates student clubs, workshops, sports tournaments, competitions, and cultural activities across partner educational institutions.

Before this web application was developed, CampusHub managed its daily work manually. The team used separate spreadsheets, email threads, chat groups, and physical paper sign-up sheets to keep track of operations. As student participation grew, several operational difficulties arose:

- **Registration Issues:** Tracking student registrations manually led to frequent errors, duplicate bookings, and missed seat allocations.
- **Scattered Information:** Event notices and updates were spread across various social media channels, leading to students missing key deadlines.
- **Disorganised Media Storage:** Event photographs and videos were kept across personal drives and private chat groups instead of a central gallery.
- **High Administrative Workload:** Staff spent significant time manually cross-checking student details, tracking attendance, and updating club rosters.

1.2 Purpose of the Web Information System

To solve these challenges, management decided to introduce **CampusHub**, a centralised web-based information system. The system brings all activities into a single digital platform designed for two main groups of users:

- **Students:** Can browse upcoming events, view announcements, register for events online, update their member profile, upload activity photographs, and access multimedia content directly.
- **Administrators:** Can manage student records, publish and update events, track registrations, upload promotional media, post announcements, and maintain organisation data efficiently.

1.3 System Overview & Technology Stack

The CampusHub web application was developed using core web technologies covered in the Web Programming I module:

- **Front-End (HTML5 & CSS3):** HTML5 provides semantic page structures and forms, while an external CSS3 stylesheet delivers clean, responsive styling across desktop and mobile screens.
- **Back-End (PHP 8.x):** Handles server-side business logic, user authentication, dynamic content rendering, form handling, and input validation.
- **Database (MySQL / MariaDB via XAMPP):** Stores structured records for users, events, and registrations, connecting to PHP through mysqli with prepared statements for secure data handling.
- **Data Exchange (XML):** Used for structured data handling and exporting event information.

- **Multimedia & File Management:** Includes audio/video integration and a file upload mechanism to store and display student media safely on the web server.

1.4 Project Objectives

The main objectives of developing this web application are:

1. Replace manual, paper-based, and spreadsheet workflows with a reliable web platform.
2. Provide an intuitive interface where students can register for events and manage their profiles independently.
3. Give administrators full control to manage events, registrations, media, and site announcements in one place.
4. Implement practical web development concepts including HTML structure, CSS layouts, PHP backend processing, database CRUD operations, XML usage, and error handling.

2. Requirements Analysis

2.1 User Identification and Stakeholders

The CampusHub system is designed around distinct user roles with specific permissions and responsibilities to keep operations organised and secure:

1. **Guest / Public Visitor:**
    - Unauthenticated visitors or new campus students exploring the platform.
    - Can browse the public home page, read the About page, view active announcements, inspect public media galleries, check upcoming event details, and submit inquiries through the contact page.
    - Must create an account or log in before registering for any event or submitting photos.
2. **Registered Student:**
    - Authenticated students who have registered an account and logged in.
    - Can access the dedicated Student Dashboard (student/dashboard.php).
    - Can browse event schedules, check seat availability, and register for events online.
    - Can view their active and past registrations under "My Registrations" (student/my_registrations.php).
    - Can manage their personal profile details, update contact numbers, and upload profile pictures (student/profile.php).

- Can upload event activity photos to the student media showcase (student/upload_photo.php).

- Can apply for open student club leadership and coordinator roles (student/apply_role.php).

3. **System Administrator (Admin / Super Admin / Moderator):**

- Authorised management personnel with administrative privileges.

- Can access the centralised Admin Dashboard (admin/dashboard.php).

- Can manage event records: create, edit, delete, publish, or archive campus events (admin/manage_events.php, admin/add_event.php, admin/edit_event.php).

- Can manage student accounts, update student records, and toggle user account statuses (admin/manage_students.php, admin/edit_student.php).

- Can oversee event participation rosters and verify registration lists (admin/manage_registrations.php).

- Can publish, update, and prioritize notices on the campus notice board (admin/manage_announcements.php).

- Can upload official media (promotional videos and event photos) to the multimedia gallery (admin/upload_media.php).

- Can export event data to structured XML files for data backups and external reporting (admin/export_events.php).

- Can update and maintain organizational contact information (admin/organization_info.php).

2.2 Functional Requirements

The functional requirements define the specific features, actions, and services the web application must provide:

1. User Account & Authentication Management

- **Student Registration:** New students can register by providing their first name, last name, phone number, email address, gender, and secure password.

- **User Login & Session Handling:** Registered users can log in using their email/phone and password. The system initiates a secure PHP session to track login state across pages.

- **Role-Based Access Control - RBAC:** The application validates session roles on protected pages (student, admin, super_admin, moderator). Unauthorized requests are redirected to appropriate landing or login pages.

- **Profile Management:** Logged-in students can update their profile information, phone numbers, and profile avatars.

- **Password Reset:** Users can request and complete a password reset workflow if they misplace their login credentials.

2. Event Management & Online Booking

- **Event Discovery & Search:** Students and visitors can browse published events, filter activities by category (e.g., workshops, sports, cultural), and search by keyword.

- **Online Event Registration:** Logged-in students can register for available events with one click. The system automatically verifies seat limits and prevents duplicate registrations for the same user.

- **Admin Event CRUD Operations:** Administrators can insert new events with banner images, update event details, toggle event visibility (display/hide), and delete canceled or expired events.

- **Registration Tracking:** Administrators can view participant rosters for each event and export or review attendance data.

3. Multimedia Showcase & File Uploading

- **File Uploading:** Students and administrators can upload event pictures and media files with accompanying titles, descriptions, and category tags (assets/images/).

- **Dynamic Gallery & Media Player:** The system dynamically displays uploaded photo albums and embeds audio/video players for campus event recaps and promotional clips.

4. Announcements & Communication

- **Announcement Board:** Administrators can publish important updates and notifications tagged by priority (low, medium, high, urgent).

- **Public Contact Form:** Visitors can send messages through a validated contact form stored for administrative review.

5. XML Data Handling

- **XML Export:** The system provides an automated feature for administrators to export complete event records into structured, valid XML format for reporting and external data exchange.

2.3 Non-Functional Requirements

The non-functional requirements define the quality attributes, usability standards, and security constraints of the web application:

- **Usability & Interface Clarity:** The user interface follows a clean layout with accessible form inputs, clear buttons, intuitive navigation bars, and informative status/error alerts for user feedback.

- **Responsive Design:** Using standard CSS flexbox and grid layouts, all pages adapt seamlessly across desktop monitors, laptops, tablets, and mobile smartphones.

- **Security & Safe Data Handling:**

- User passwords are encrypted using PHP's password_hash() (Bcrypt).
- Database operations utilize prepared statements with parameter binding (mysqli_stmt_bind_param) to prevent SQL Injection attacks.
- All dynamic user outputs rendered in HTML are sanitized using htmlspecialchars() to mitigate Cross-Site Scripting (XSS).

- **Data Integrity:** Foreign key constraints between relational tables (users, events, registrations, media) maintain referential integrity and prevent orphaned records upon data updates or deletions.
- **Performance & Reliability:** The backend runs efficient procedural PHP scripts and optimized SQL queries, ensuring fast page load speeds on local Apache/XAMPP test environments.

2.4 Documented Assumptions

In accordance with the assignment guidelines, the following technical and operational assumptions were made during development:

1. **Development & Server Environment:** The web application is deployed and tested locally using the XAMPP stack (Apache Web Server, PHP 8, and MySQL database on default port 3306).
2. **Client Accessibility:** End users access the web application through modern, standard web browsers (Google Chrome, Mozilla Firefox, Microsoft Edge) with JavaScript and cookies enabled.
3. **User Identification:** Every registered student possesses a unique email address and contact telephone number used for account identification and authentication.
4. **Local File Storage:** Uploaded files (JPEG, PNG, WebP image formats, and MP4 video formats) are stored in server-side directories (assets/images/, assets/videos/), while their relative file path references and metadata are stored in the MySQL database.
5. **Database Seeding:** The system is pre-configured with default seed data—including an initial administrator account, sample event categories, announcements, and demo events—to facilitate comprehensive testing and live demonstration.

3. Website Planning

3.1 Website Purpose and Scope

The main purpose of the CampusHub web application is to centralise and automate student activity management, event participation, and communication across campus communities.

The system scope is organised into three distinct operational areas:

- **Public Portal:** Open to all visitors and students to explore campus updates, view published notices, check the event directory, view multimedia galleries, and submit general inquiries.

- **Student Member Portal:** A secure area where authenticated students can update their profiles, register for upcoming events, monitor their bookings, submit activity photographs, and apply for student roles.

- **Administrative Portal:** A management dashboard where authorized staff can schedule events, oversee student records, review registration lists, publish announcements, manage media, and export event data to XML.

3.2 Navigation Architecture & Sitemap

The website is structured logically to ensure intuitive navigation for all users. The site map below illustrates the hierarchy and file organisation across public, student, and admin areas:

CampusHub Web Application

- Public Pages
  - Home (index.php)
  - Events (events.php)
    - Event Details (event_details.php)
    - Register for Event (register_event.php)
  - Announcements (announcements.php)
    - Announcement Details (announcement_details.php)
  - Gallery (gallery.php)
  - About Us (about.php)
  - Contact Us (contact.php)
  - Login (login.php)
  - Register (register.php)
  - Forgot Password (forgot_password.php)
- Student Portal (student/)
  - Dashboard (student/dashboard.php)
  - My Profile & Avatar (student/profile.php)
  - My Registrations (student/my_registrations.php)
  - Upload Activity Photo (student/upload_photo.php)
  - Apply for Club Role (student/apply_role.php)
- Administrator Portal (admin/)
  - Admin Dashboard (admin/dashboard.php)
  - Manage Events (admin/manage_events.php)
    - Add Event (admin/add_event.php)
    - Edit Event (admin/edit_event.php)
    - Delete Event (admin/delete_event.php)
  - Manage Students (admin/manage_students.php)
    - Edit Student (admin/edit_student.php)
  - Manage Registrations (admin/manage_registrations.php)
  - Manage Announcements (admin/manage_announcements.php)
  - Upload Gallery Media (admin/upload_media.php)
  - Manage User Roles (admin/manage_roles.php)
  - Export Events to XML (admin/export_events.php)
  - Organization Settings (admin/organization_info.php)

3.3 Key Design Decisions

1. Modular PHP Include Architecture
To eliminate repetitive code and simplify maintenance across more than twenty pages, I used PHP modular includes:

- includes/header.php: Contains standard HTML <head> tags, CSS links, branding logos, responsive navigation links that update dynamically based on user session status, and mobile menu controls.
- includes/footer.php: Contains site navigation links, copyright details, organization contact info, and script inclusions.
- includes/db.php: Manages the MySQL database connection using PHP's mysqli extension and handles connection error handling.

2. Visual Design System & Color Palette
A student-friendly and accessible color scheme was selected to maintain visual consistency across all pages:

- **Primary Action Color (**#FF6B4A **- Coral / Warm Orange):** Used for primary buttons, active navigation indicators, and key calls-to-action to draw student focus.
- **Base Text & Navigation Dark Tone (**#171B2A / #0F172A **- Slate):** Provides strong visual hierarchy and high readability for body text and navigation bars.
- **Background Surfaces (**#F8FAFC **and** #FFFFFF **- Clean White & Soft Gray):** Keeps content cards and forms distinct, clean, and readable.
- **Status & Feedback Colors:**
    - Success (#10B981 - Emerald Green): For successful registrations and active badges.
    - Warning (#F59E0B - Amber): For high-priority notices and low-seat warnings.
    - Danger (#EF4444 - Rose Red*):* For error messages and deletion confirmations.

3. Typography Selection
The interface uses modern system fonts (-apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif). This provides clean, native font rendering across Windows, macOS, Android, and iOS while avoiding external font download delays.

4. Layout & Grid Strategy
The application utilizes pure **CSS Flexbox** and **CSS Grid** for structure rather than heavy external UI frameworks. This ensures lightweight page weights, fast rendering in local environments, and practical demonstration of core CSS concepts required in Web Programming I.
4. User Interface Design
4.1 UI Design Philosophy

The user interface (UI) for CampusHub was built on three main ideas: simplicity, clarity, and responsiveness. Because students and staff will access the system using different devices like mobile phones, laptops, and university lab computers, the design automatically adjusts to fit any screen size. The content presentation is kept clean so users can find what they need without confusion.

4.2 Header and Navigation Structure

The header is the main navigation tool for the entire web application, fulfilling the requirement for a clear navigation structure. It includes:

- **Brand Logo & Title:** A clean logo with the text "CampusHub" that links directly back to the homepage.
- **Main Navigation Links:** Simple links to the Home, Events, Announcements, Gallery, About, and Contact pages.
- **Dynamic User Action Area:** The menus change depending on who is using the site:
    - For Guests*:* Shows "Login" and "Register" buttons.
    - For Logged-in Students*:* Shows a "Dashboard" button, a profile picture, and a "Logout" link.
    - For Administrators*:* Shows an "Admin Panel" shortcut with a special badge.
- **Mobile Menu:** On smaller screens, the links collapse into a hamburger menu to save space.

4.3 Footer Component

The footer provides a clean ending at the bottom of every page:

- **Column 1:** A short summary of CampusHub and its mission.
- **Column 2:** Quick links to the most used pages, like Events and Gallery.
- **Column 3:** Contact details for the student help center and emergency helplines.
- **Bottom Bar:** A copyright notice with the current year generated by PHP.

4.4 Major Page Layouts

To ensure clear content presentation, the main pages were designed with specific layouts:

1. Home Page (index.php**)**

- **Hero Section:** A large banner at the top with a clear call-to-action to "Explore Events."
- **Metrics Counter:** A section showing the total number of registered students and active clubs.

- **Featured Events:** A responsive 3-column grid showing upcoming events with images and "Register" buttons.
- **Announcements Preview:** A section highlighting important notices so students see them immediately.

2. Events Directory & Event Details (events.php **&** event_details.php**)**

- **Search & Filter:** A bar that lets students search for events by keyword or filter by categories like Tech or Sports.
- **Event Cards:** Small boxes for each event showing a picture, date, location, and available seats.
- **Event Detail Page:** A full page showing the complete description, schedule, and a one-click registration form.

3. Multimedia Gallery (gallery.php**)**

- **Filters:** Buttons to quickly sort media by Photos or Videos.
- **Responsive Grid:** A layout of cards showing image previews and the name of the person who uploaded them.
- **Interactive Lightbox:** Clicking an image opens it in a full-screen popup so users can view it clearly.

4. Student Dashboard (student/dashboard.php**)**

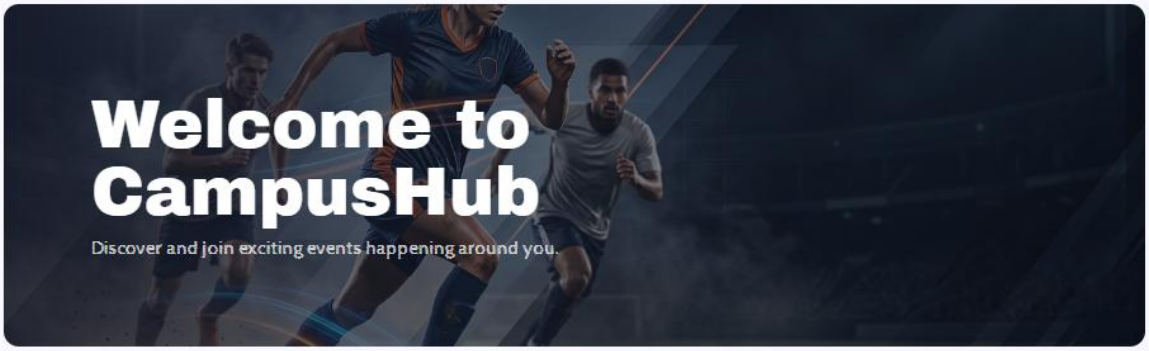
- **Welcome Banner:** Greets the student by their first name and shows their current role.
- **Summary Cards:** Quick numbers showing their active registrations and uploaded photos.
- **Action Shortcuts:** Fast links to update their profile or view registered events.

5. Administrator Dashboard (admin/dashboard.php**)**

- **Metric Grid:** Live numbers showing total students, events, and registrations.
- **Action Hub:** Quick buttons to manage events, manage students, publish announcements, and export XML reports.
- **Recent Activity:** A simple table showing the newest student signups and event bookings.

4.5 UI Layout Wireframe Structure

Home About Dashboard Events Students Registrations Notices Gallery



Featured Events

Find your next campus experience.

Showing 2 upcoming events



AUG 27, 2026 · 4:40 PM

test

test



SEP 08, 2026 · 9:00 AM

Cricket match

test two

1 Browse All Events »

Latest Announcements

Official circulars from Student Services.

View All »

test

test

AUG 26, 2026

CampusHub

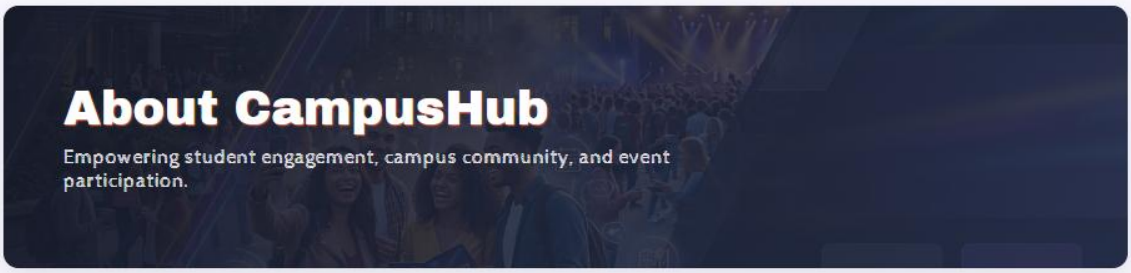
Your campus, connected. Discover events, share moments, and stay in the loop.

EXPLORE

Home
About
Announcements
Contact

PORTALS

Campus Events
Student Portal
Admin Portal



About CampusHub

Empowering student engagement, campus community, and event participation.

Our Mission

CampusHub was engineered as a comprehensive digital portal for student affairs, designed to bridge the communication gap between university academic councils, student clubs, and attendees. We provide seamless event scheduling, transparent registration passes, and prompt administrative notices.

Our Vision

To cultivate an energized, connected, and collaborative campus ecosystem where every student has immediate access to extracurricular enrichment, technical workshops, sports championships, and cultural celebrations throughout their academic journey.

CampusHub by the Numbers

Our growing community across academic faculties.

500+	45+	12+	98%
Enrolled Students	Events & Workshops	Active Student Clubs	Participation Rate

Core Student Services

Key features available through the student portal.

Technical Workshops

Hands-on coding bootcamps, AI hackathons, and software engineering sessions run by computing societies and guest industry mentors.

Sports & Tournaments

Annual inter-faculty championships, badminton leagues, cricket tournaments, and athletics meets managed by the Sports Council.

Official Announcements

Instant official notices from the Student Affairs Division regarding semester schedules, deadlines, logistics, and emergency alerts.

Academic Institution Information

Developed for **Java Institute for Advanced Technology** student services, CampusHub demonstrates complete web development practices including dynamic relational database queries, authenticated session management, server-side data validation, and clean responsive user interfaces.

Home About Dashboard Events Students Registrations Notices Gallery

Event Management

Add, edit, and maintain active campus events.

+ Add New Event

Event Title	Date & Time	End-time	Seats	Status	Actions
Cricket match Sports • galle stadium	Sep 08, 2026 9:00 AM	6:30 PM	150	Display	Edit Delete
test General • Campus Grounds	Aug 27, 2026 4:40 PM	10:40 PM	100	Display	Edit Delete

CampusHub

Your campus, connected. Discover events, share moments, and stay in the loop.

EXPLORE

Home
About
Announcements
Contact

PORTALS

Campus Events
Student Portal
Admin Portal

© 2026 CampusHub. All rights reserved.

Home About Dashboard Events Students Registrations Notices Gallery

Registered Students Directory

View student user accounts, academic IDs, and campus engagement records.

Student ID	Full Name	Email Address	Contact Phone	Gender	Institution	Event Passes
STU-0007	admin test	admin@gmail.com	+940762266991	Female	CampusHub Campus	1 Events
STU-0003	moderator test	modarator@gmail.com	+940112233445	Male	CampusHub Campus	0 Events
STU-0002	BANUJA KAUSHALYA	btv731563@gmail.com	+94767868297	Male	CampusHub Campus	0 Events
STU-0001	test test	test@gmail.com	+941234567890	Male	CampusHub Campus	0 Events

CampusHub

Your campus, connected. Discover events, share moments, and stay in the loop.

EXPLORE

Home
About
Announcements
Contact

PORTALS

Campus Events
Student Portal
Admin Portal

© 2026 CampusHub. All rights reserved.

Home About Dashboard Events Students Registrations Notices Gallery

Campus Gallery & Memories

Explore event highlights, student showcases, workshops, and sports championships across campus.

Upload Media (Admin/Mod)

Filter & Sort

KEYWORD SEARCH

Search title, details, tags...

SORT GALLERY

Newest Uploads First

ASSOCIATED EVENT

All Events & General

MEDIA FORMAT

All Photos Videos

Apply Filters

Showing **1** of 1 media items



Cricket match

test

test

Campus Gallery test test

CampusHub

Your campus, connected. Discover events, share moments, and stay in the loop.

EXPLORE

Home
About
Announcements
Contact

PORTALS

Campus Events
Student Portal
Admin Portal

© 2026 CampusHub. All rights reserved.

Admin Command Center

Manage campus events, student registrations, official circulars & system settings.

4	Total Students
2	Total Events
1	Registrations
1	Announcements

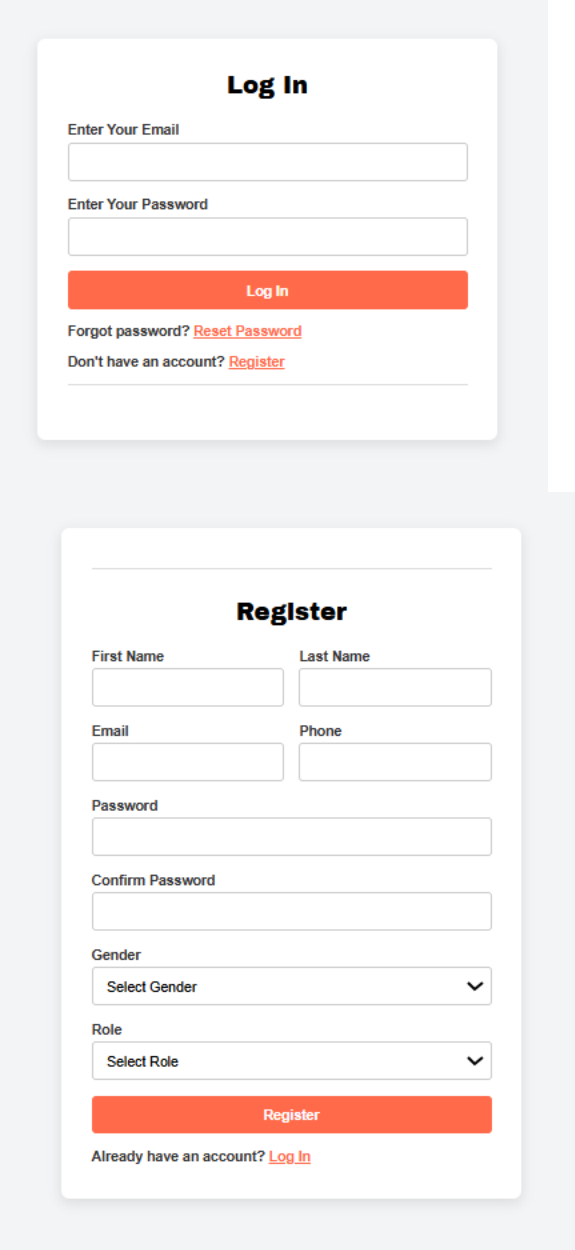
- Manage Events
- Add New Event
- Upload Media
- Student Directory
- Registrations
- Announcements
- Roles & Approvals
- Org Info
- Master Control

Latest Student Event Registrations

View All »

Student	ID	Event	Registration Date	Status
admin test admin@gmail.com	REG-0001	Cricket match	Aug 27, 2026 · 4:55 PM	CONFIRMED

© 2026 CampusHub. All rights reserved.



5. HTML and CSS Development

5.1 HTML5 Implementation Approach

I used standard HTML5 elements to develop both the static and dynamic web pages for CampusHub. By using semantic tags, I ensured the codebase is well-structured, easy to read, and properly organized.

The key semantic HTML tags I implemented include:

- <header>: Placed at the top of every page to hold the site branding and navigation.
- <nav>: Used to wrap the primary menu links so users can easily move around the site.
- <main>: Wraps the unique central content on each individual page.

- <section>: Used to divide the pages into clear areas, like the hero banner, statistics widgets, and featured event grids.

- <article>: Used for self-contained blocks of content, such as individual event cards, notice board announcements, and gallery photos.

- <form>, <input>, <select>, <button>: Used to build clear and accessible user input forms for logins and registrations.

HTML Code Snippet: Semantic Event Card

HTML

```
<article class="event-card">
  <div class="event-thumb">
    <img src="assets/images/hackathon.jpg" alt="Campus Hackathon 2026">
    <span class="category-badge">Technology</span>
  </div>
  <div class="event-content">
    <time datetime="2026-10-15" class="event-date">Oct 15, 2026</time>
    <h3 class="event-title">Campus Hackathon 2026</h3>
    <p class="event-summary">Join the 24-hour coding challenge at the main auditorium.</p>
    <div class="event-footer">
      <span class="seat-badge">45 Seats Left</span>
      <a href="event_details.php?id=1" class="btn btn-primary">View Details</a>
    </div>
  </div>
</article>
```

5.2 CSS3 Implementation & Styling Approach

To keep the design clean and consistent, I applied external CSS, using a main stylesheet (assets/css/style.css) linked to all pages. This allowed me to use reusable styling techniques and advanced selectors throughout the web application.

My main CSS implementation techniques include:

1. **CSS Custom Properties (Variables):** I defined global root variables for colors (like the primary coral orange and clean white backgrounds) and spacing. This makes it easy to update the theme in one place.

2. **Layout Design with Grid and Flexbox:** I used CSS Grid to build the responsive 3-column event catalogues and media galleries. I used Flexbox to align items perfectly inside the navigation bars, form button groups, and dashboard statistic cards.

3. **Hover Effects and Micro-Interactions:** I applied simple CSS transitions to buttons and event cards. When a user hovers over an event card, it gently floats up with a subtle drop shadow to make the interface feel interactive.

4. **Responsive Media Queries:** I wrote media queries to ensure the formatting and layout design adapt automatically. For example, the 3-column grid changes to a single column on mobile phones, and the main menu turns into a mobile toggle button.

CSS Code Snippet: Grid Layout and Hover Effects

CSS

```
/* Reusable Styling Variables */
:root {
  --primary-color: #FF6B4A;
  --shadow-hover: 0 10px 25px rgba(0,0,0,0.12);
}

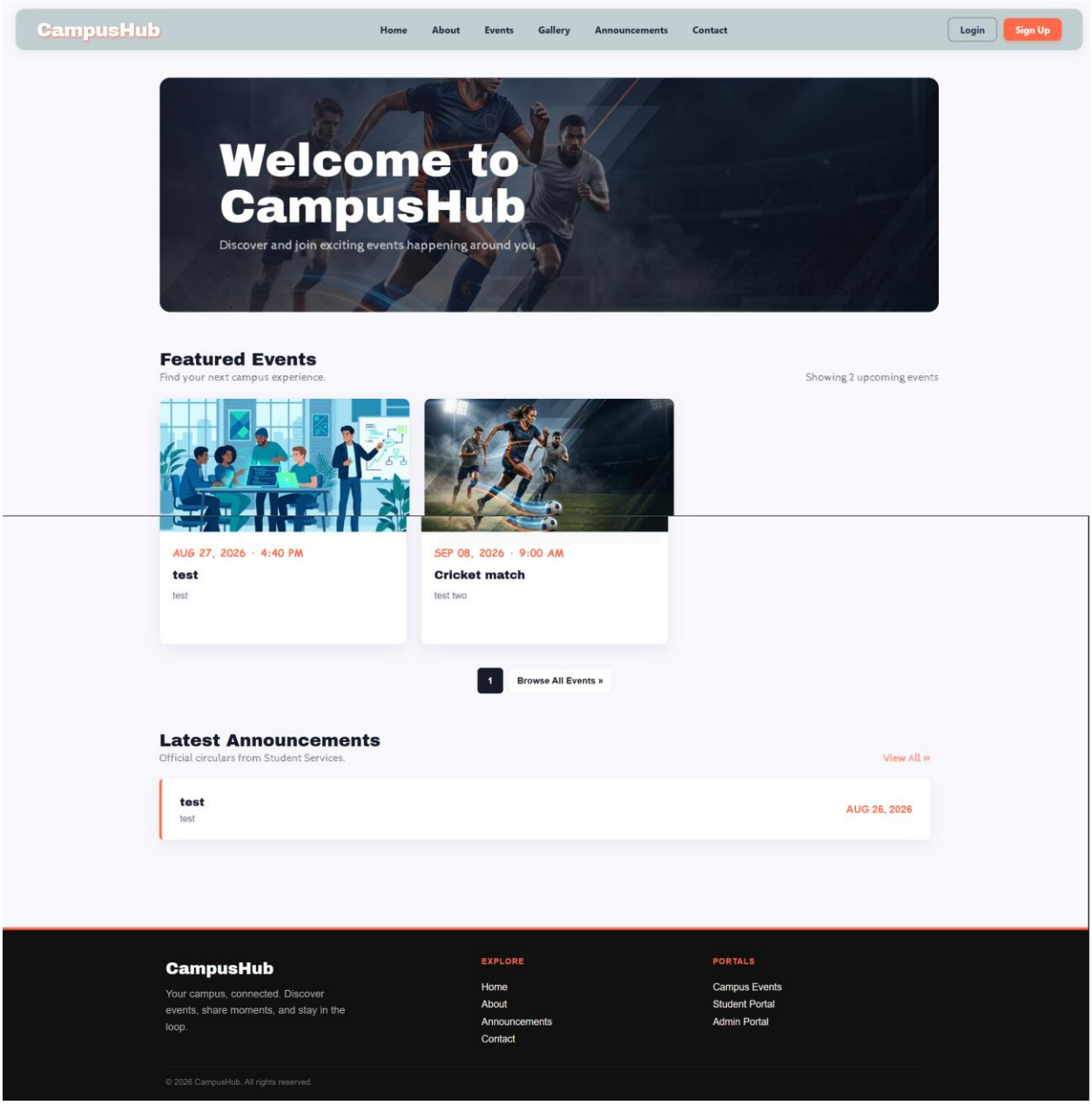
/* Grid Layout for Events */
.events-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(300px, 1fr));
  gap: 24px;
}

/* Hover Animation technique */
.event-card {
  transition: transform 0.25s ease, box-shadow 0.25s ease;
}
.event-card:hover {
  transform: translateY(-5px);
  box-shadow: var(--shadow-hover);
}

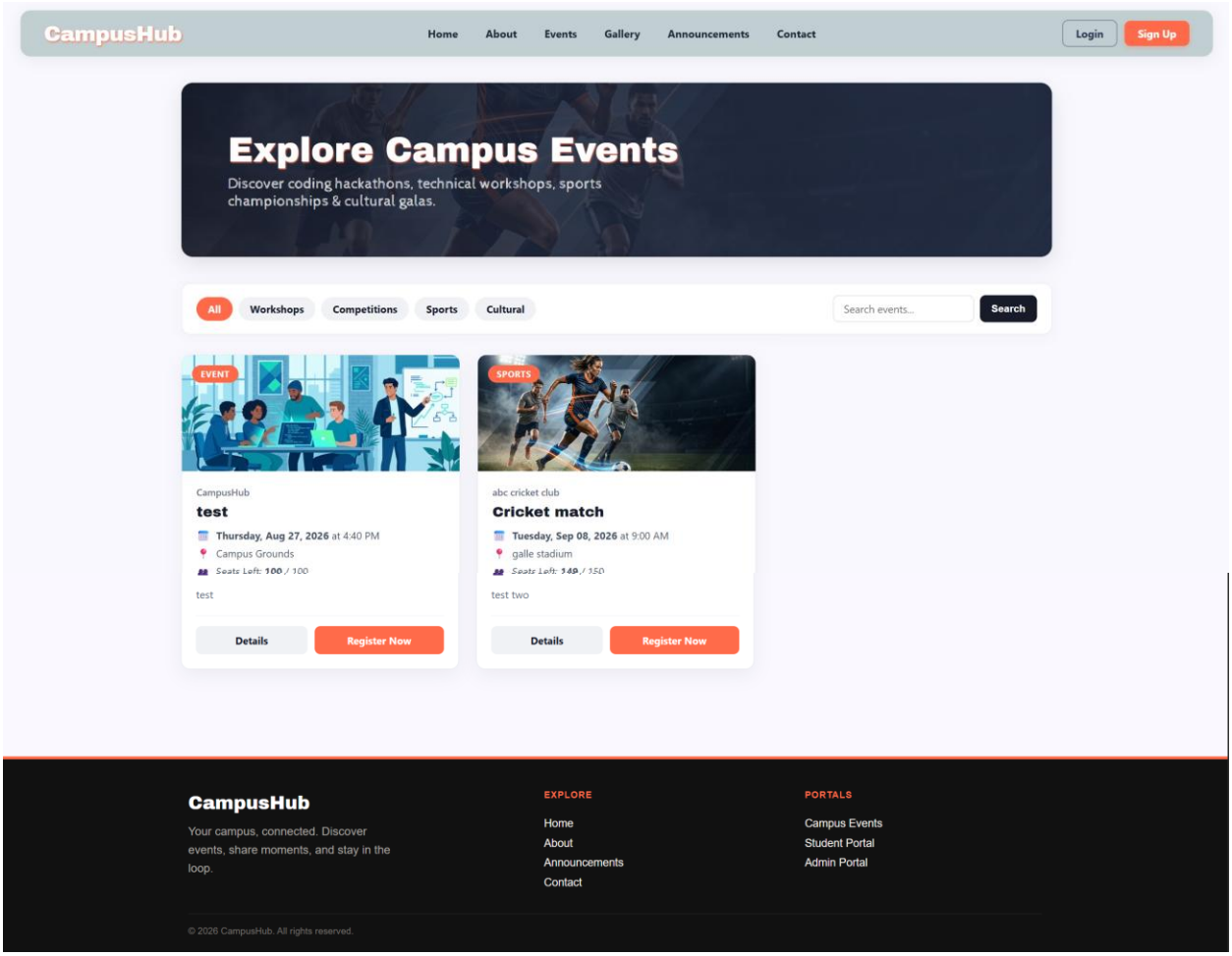
/* Mobile Responsive Formatting */
@media (max-width: 768px) {
  .events-grid {
    grid-template-columns: 1fr;
  }
}
```

5.3 Implementation Screenshots

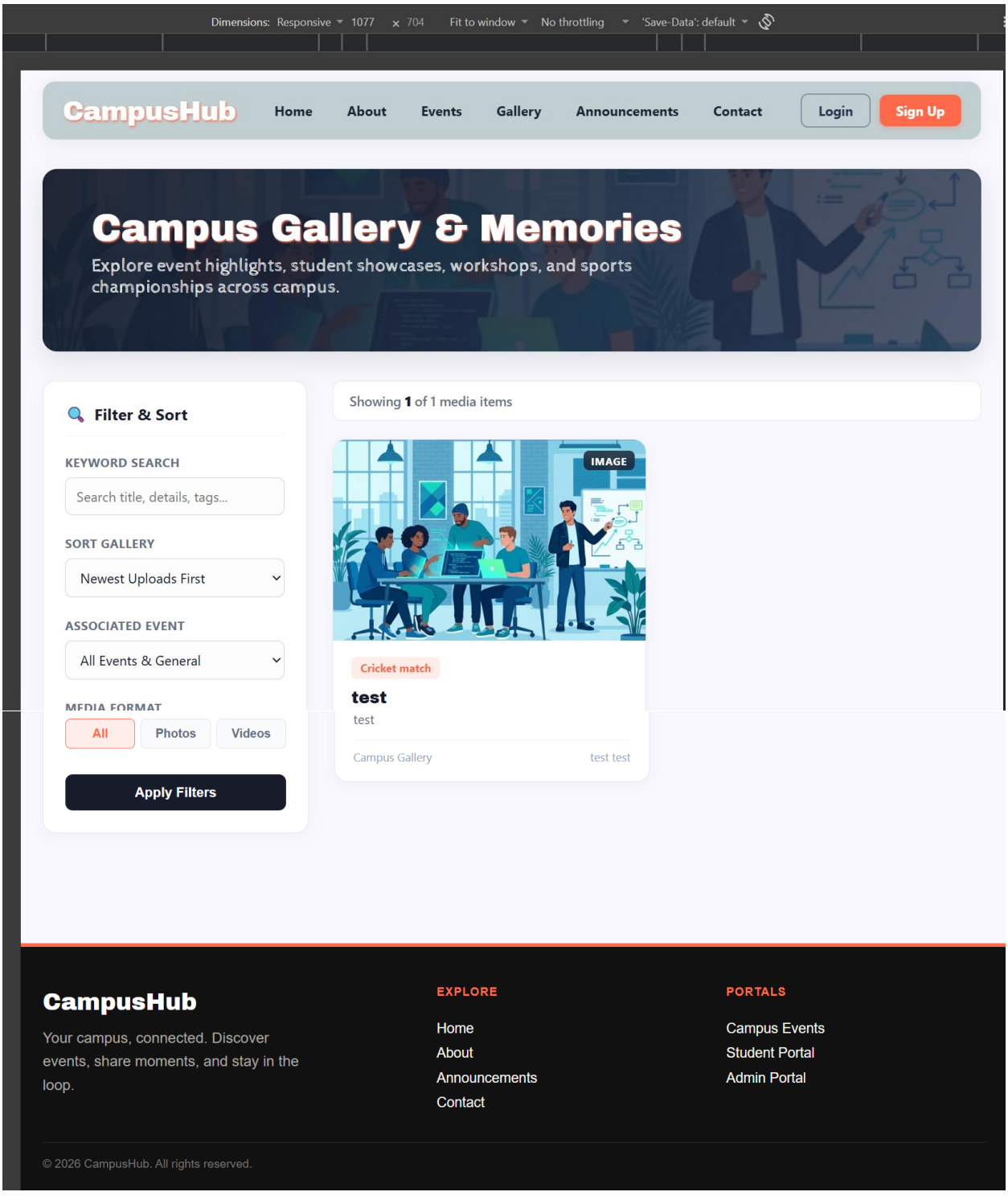
CampusHub Homepage



Public Event Directory



Responsive Media Gallery



5. PHP Development

6.1 Server-Side Architecture Overview

To make the CampusHub web application dynamic, I used PHP as the main backend language. PHP acts as the engine of the website. It processes the information that users type into forms, communicates securely with the database, checks if a user is a student or an admin, and builds the web pages dynamically before sending them to the user's browser.

6.2 PHP Variables and Data Types
Following the project requirements, I used different PHP variables and data types to store and manage information:

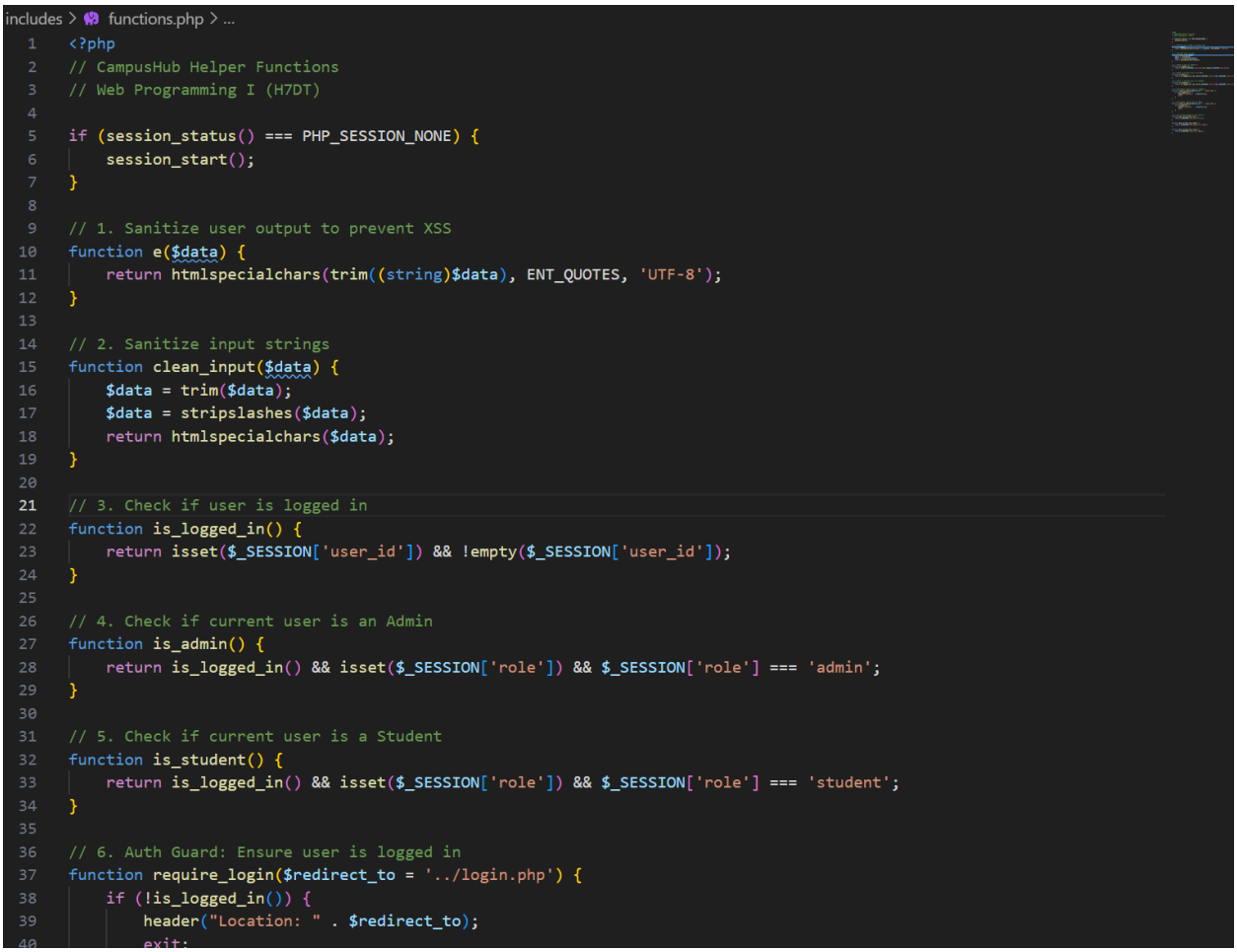
- **Strings:** Used to store text, like user names and email addresses (e.g., $first_name = "Alex";).
- **Integers:** Used for numbers, like database IDs and counting available event seats (e.g., $event_id = 3;).
- **Arrays:** Used to hold lists of information. I used both indexed arrays for lists of records and associative arrays for structured data like event details.
- **Booleans:** Used for true/false checks, like verifying if a form submission is valid ($is_valid = true;).

6.3 Modular File Inclusions
To avoid writing the same code over and over (the DRY principle - Don't Repeat Yourself), I used PHP include statements to load reusable page parts.

I put common files inside an includes/ folder and loaded them using require_once. This ensures that important files like the database connection and the website header are loaded exactly one time per page, which prevents errors.

PHP
```
<?php
// Loading reusable layout components and database connection
require_once __DIR__ . '/../includes/db.php';
require_once __DIR__ . '/../includes/functions.php';
require_once __DIR__ . '/../includes/header.php';
?>
```
6.4 Custom Functions
I created custom PHP functions to handle repetitive tasks easily and securely. I grouped these into a helper library file called functions.php.

These functions help clean user input to stop security attacks (like XSS), check if a user is logged in, and verify user roles to protect admin pages.



PHP

```
<?php
// 1. Output encoding to prevent security attacks
function e($data) {
    return htmlspecialchars(trim((string)$data), ENT_QUOTES, 'UTF-8');
}

// 2. Input sanitization for user form submissions
function clean_input($data) {
    $data = trim($data);
    $data = stripslashes($data);
    return htmlspecialchars($data, ENT_QUOTES, 'UTF-8');
}

// 3. User authentication status check
function is_logged_in() {
    return isset($_SESSION['user_id']) && !empty($_SESSION['user_id']);
}
?>
```

6.5 Form Handling & Parameter Passing

To make the application interactive, I successfully implemented HTML forms and PHP parameter passing.

1. Form Processing ($_POST**)** When a user submits private information, like registering for an event or logging in, I used the POST method. The PHP code checks if the form was submitted, reads the data, and connects to the database to save it.

PHP

```
<?php
// Example: Processing an event registration form safely
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['register_event'])) {
   $event_id = isset($_POST['event_id']) ? (int)$_POST['event_id'] : 0;
   $user_id  = $_SESSION['user_id'] ?? 0;

   // Database insert logic goes here...
}
?>
```

2. Parameter Passing ($_GET**)** For public actions like searching or viewing specific pages, I passed parameters through the URL using the GET method.

- event_details.php?id=3: The id=3 parameter tells PHP to load details for event number 3.
- events.php?category=Sports: Tells PHP to filter the event list to show only sports activities.

6.6 Session Management

To keep users logged in as they move from page to page, I used PHP sessions. By calling session_start() at the top of my scripts, the server remembers the user.

When a student logs in, the system stores their user ID, name, and role in $_SESSION variables. This is what allows the header to change and say "Welcome" instead of showing a Login button. When the user clicks Logout, the system safely destroys the session data.

7.Database Connectivity and Data Manipulation

7.1 Database Architecture and Schema Design

The CampusHub application is powered by a relational MySQL database designed to safely store users, events, and registration data. The database schema uses relationships between tables to ensure data stays organized and accurate.

Main Database Tables:

1. users **Table:** Stores all account information including names, emails, phone numbers, encrypted passwords, user roles (student or admin), and profile pictures.
2. events **Table:** Stores all the campus activities, including the event title, description, date, time, available seats, and the promotional image path.
3. registrations **Table:** This table acts as a bridge. It links a specific user_id to a specific event_id when a student books a ticket, recording the exact date of registration.

4. media **Table:** Stores the file paths and descriptions for photographs and videos uploaded by users.

5. announcements **Table:** Holds the important notices published by administrators to display on the notice board.

7.2 Database Connection

To connect the PHP web application to the MySQL database, I used PHP's mysqli extension and created a centralized connection file (includes/db.php). I wrote a custom helper class to manage the connection safely and prevent common security issues.

PHP

```
<?php
// includes/db.php - Centralized Database Connection
class Database {
   private static $connection = null;

   // Connect to MySQL running on XAMPP (port 3306)
   public static function getConnection() {
      if (self::$connection === null) {
         $conn = @new mysqli("localhost", "root", "", "campushub", 3306);
         if ($conn->connect_error) {
            return null;
         }
         $conn->set_charset("utf8mb4");
         self::$connection = $conn;
      }
      return self::$connection;
   }

   // ... Helper functions for executing queries safely ...
}
?>
```

7.3 Data Manipulation (CRUD Operations)

The system performs full CRUD (Create, Read, Update, Delete) operations using SQL queries and PHP prepared statements to protect against SQL injection attacks.

1. CREATE (Insert Operation) When a student fills out the registration form, the system creates a new record in the database:

PHP

```
// Creating a new user record safely with prepared statements
$hashedPassword = password_hash($password, PASSWORD_DEFAULT);
$query = "INSERT INTO users (first_name, last_name, phone, email, password_hash, role)
        VALUES (?, ?, ?, ?, ?, 'student')";
// Data is inserted safely using parameter binding
Database::iud($query, "sssss", [$first_name, $last_name, $phone, $email, $hashedPassword]);
```

2. READ (Select Operation) To display the list of events on the public directory page, the system reads records from the events table:

PHP
```
// Selecting active events to display on the webpage
$query = "SELECT * FROM events WHERE status = 'display' ORDER BY event_date ASC";
$result = Database::search($query);

// Looping through the records to generate HTML dynamically
while ($row = $result->fetch_assoc()) {
   echo "<h3>" . htmlspecialchars($row['title']) . "</h3>";
}
```
3. UPDATE (Update Operation) When a logged-in student updates their profile details on the dashboard, the system modifies the existing record:

PHP
```
// Updating the student's name and phone number
$query = "UPDATE users SET first_name = ?, last_name = ?, phone = ? WHERE user_id = ?";
Database::iud($query, "sssi", [$first_name, $last_name, $phone, $user_id]);
```
4. DELETE (Delete Operation) When an administrator needs to remove a canceled event from the platform, the system deletes the record entirely:
PHP
```
// Deleting an event based on its ID
$event_id = (int)$_GET['id'];
$query = "DELETE FROM events WHERE event_id = ?";
Database::iud($query, "i", [$event_id]);
```
8. XML Implementation

8.1 The Role of XML in the System

To meet the assessment requirements, I needed to demonstrate the use of XML technologies within the web application. Extensible Markup Language (XML) is a standard format used to store and transport data.

In the CampusHub project, I used XML to create a data export feature for administrators. This allows staff to click a button and download the entire schedule of campus events into a structured XML file, which can then be used for backups or opened in spreadsheet software.

8.2 XML Export Implementation

I built this feature in the admin panel (admin/export_events.php). To generate the XML file, I used PHP's built-in SimpleXMLElement function.

How it works:

1. When an administrator clicks "Download XML Export," the PHP script starts running.
2. The script sends special headers to the browser so it knows to download the file instead of displaying a web page.

3. It creates a main root tag called <events>.

4. It connects to the database, reads all the event records, and loops through them. For every event, it adds a new <event> child node along with its details like title, date, time, and available seats.

5. Finally, it prints the XML string so the browser can download it.

PHP Code Snippet for XML Generation:

PHP

```php
<?php
// Creating the root XML element
$xml = new SimpleXMLElement('<?xml version="1.0" encoding="UTF-8"?><events></events>');

// Query event records from the database
$res = Database::search("SELECT * FROM events ORDER BY event_date ASC");

if ($res && $res->num_rows > 0) {
   while ($row = $res->fetch_assoc()) {
      // Create an event node for each row
      $eventNode = $xml->addChild('event');
      $eventNode->addChild('event_id', (string)$row['event_id']);
      $eventNode->addChild('title', htmlspecialchars((string)$row['title']));
      $eventNode->addChild('event_date', (string)$row['event_date']);
      $eventNode->addChild('seats', (string)$row['seats']);
   }
}

// Output formatted XML to download
echo $xml->asXML();
?>
```

8.3 Sample Generated XML Document

Here is an example of what the downloaded XML file looks like. It clearly shows the proper XML syntax and nested data structure required for the project.

XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<events>
  <event>
    <event_id>1</event_id>
    <title>Annual Inter-Campus Hackathon 2026</title>
    <description>A 24-hour coding marathon bringing together student developers.</description>
    <event_date>2026-10-15</event_date>
    <event_start_time>09:00:00</event_start_time>
    <event_end_time>17:00:00</event_end_time>
    <status>display</status>
    <seats>150</seats>
    <meta_data>
      <category>Technology</category>
```

```
    <location>Main Computing Complex - Lab 4</location>
   </meta_data>
  </event>
</events>
```

8.4 XML Syntax and Validation

To ensure the XML implementation is fully functional and valid, I applied strict validation rules in my code:

- **Well-Formed Structure:** I made sure every opening tag (like <event>) has a matching closing tag (</event>).
- **Safe Characters:** I used the PHP htmlspecialchars() function on user-generated text (like event titles). This stops special characters like < or & from breaking the XML format.
- **Proper Encoding:** The document uses UTF-8 encoding at the very top so it can handle different text symbols safely.

9. Multimedia and File Upload Features

9.1 Multimedia Integration

To create an engaging and interactive student experience, CampusHub includes multimedia features such as photos and promotional video playback.

1. HTML5 Video Integration I used native HTML5 multimedia elements to display video content directly in the browser without requiring external plugins or third-party players:

HTML

```
<!-- Native HTML5 Video Player -->
<video controls class="gallery-video-player" poster="assets/images/video_poster.jpg">
  <source src="assets/images/campus_tour_2026.mp4" type="video/mp4">
  Your browser does not support the video tag.
</video>
```

2. Interactive Showcase Gallery (gallery.php**)** On the public gallery page, students can browse event photos and video clips. Clicking any photo or video card launches a modal popup window, allowing students to view high-resolution media, read event captions, and see tags clearly.

9.2 File Upload Implementation

The web application includes file upload features for both students (uploading event photos) and administrators (uploading event banners and official clips).

1. HTML Upload Form Structure To enable file transmission from the browser to the web server, the HTML <form> must use the POST method and include the enctype="multipart/form-data" attribute:

HTML

```
<form action="upload_media.php" method="POST" enctype="multipart/form-data">
  <label for="title">Media Title:</label>
  <input type="text" id="title" name="title" required>
```

```
  <label for="media_type">Media Type:</label>
  <select id="media_type" name="media_type">
    <option value="image">Photo / Image</option>
    <option value="video">Video Clip</option>
  </select>

  <label for="media_file">Choose File:</label>
  <input type="file" id="media_file" name="media_file" accept="image/*,video/*" required>

  <button type="submit" name="upload_media">Upload Media</button>
</form>
```

2. Server-Side PHP Upload Handler When the form is submitted, PHP processes the file using the $_FILES superglobal array. The script validates the file, moves it to the server upload directory (assets/images/), and stores the relative path in the MySQL database:

PHP

```
<?php
require_once '../includes/db.php';

if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_FILES['media_file'])) {
    $title = trim($_POST['title'] ?? '');
    $media_type = ($_POST['media_type'] === 'video') ? 'video' : 'image';
    $event_id = (int)($_POST['event_id'] ?? 0);
    $user_id = $_SESSION['user_id'] ?? 1;

    if ($_FILES['media_file']['error'] === UPLOAD_ERR_OK) {
        $file_tmp  = $_FILES['media_file']['tmp_name'];
        $file_name = $_FILES['media_file']['name'];
        $file_size = $_FILES['media_file']['size'];

        // Extract extension and check whitelist
        $file_ext = strtolower(pathinfo($file_name, PATHINFO_EXTENSION));
        $allowed_exts = ['jpg', 'jpeg', 'png', 'webp', 'mp4'];

        if (in_array($file_ext, $allowed_exts) && $file_size <= 10 * 1024 * 1024) {
            // Generate a unique file name to prevent accidental overwriting
            $unique_name = time() . '_' . preg_replace('/[^a-zA-Z0-9_\.-]/', '_', $file_name);
            $target_path = __DIR__ . "/../assets/images/" . $unique_name;

            // Move file from temporary directory to server folder
            if (move_uploaded_file($file_tmp, $target_path)) {
                $meta_json = json_encode(['description' => htmlspecialchars($_POST['description'] ?? '')]);

                // Save file metadata into the database
                Database::iud(
                    "INSERT INTO media (title, media_type, meta_data, events_event_id, users_user_id,
file_path) VALUES (?, ?, ?, ?, ?, ?)",
                    "sssiis",
                    [$title, $media_type, $meta_json, $event_id, $user_id, $unique_name]
                );
```

```
        }
      }
    }
}
?>
```

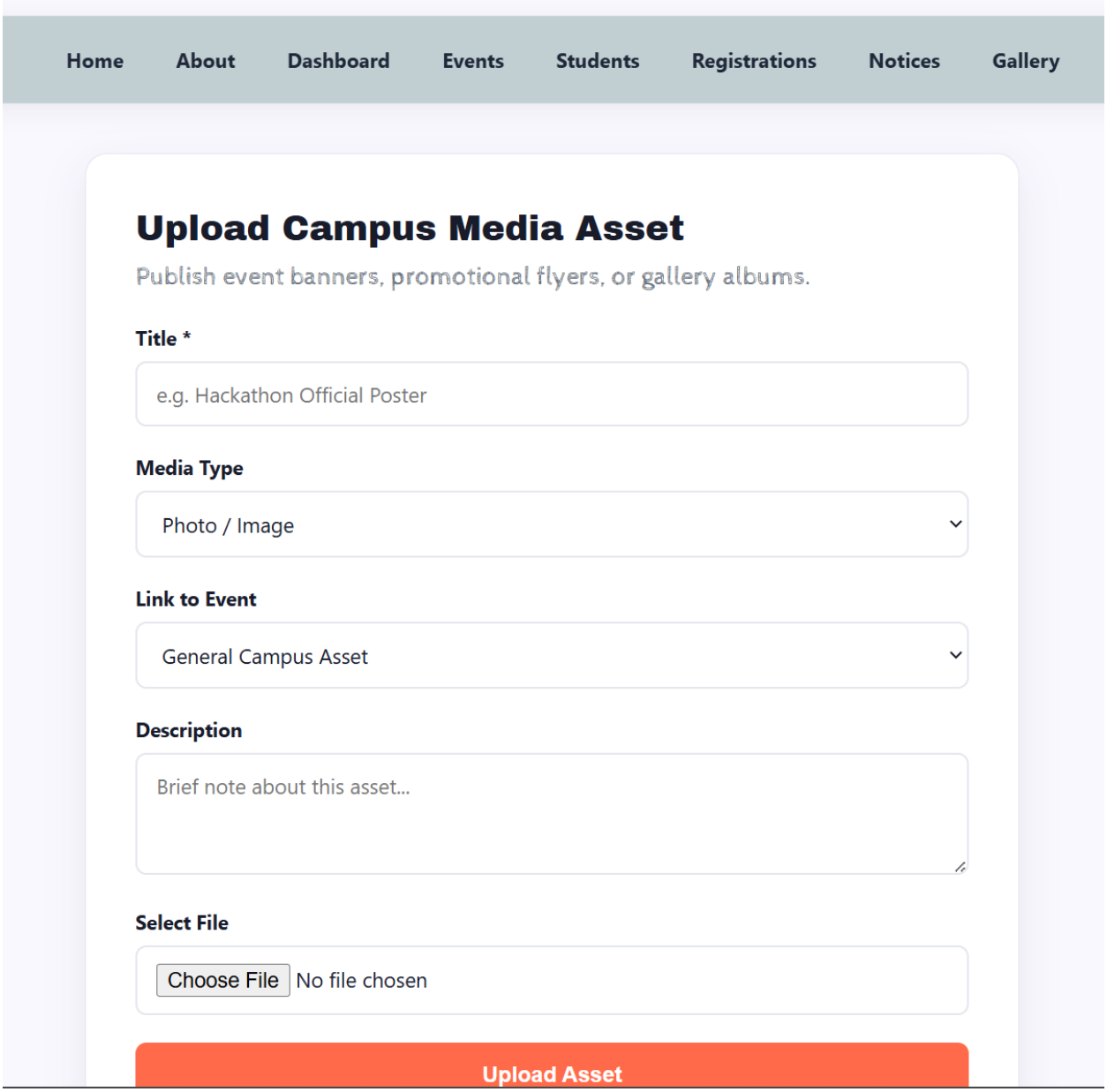
9.3 Security Measures for File Uploads

To prevent unauthorized or dangerous files from being uploaded to the web server, I implemented several security checks:

- **File Extension Whitelisting:** The PHP script strictly allows safe image and video formats (.jpg, .jpeg, .png, .webp, .mp4). Dangerous executable formats (such as .php, .exe, .sh) are rejected immediately.
- **Unique Filename Generation:** Prepending a PHP time() timestamp ensures that uploaded files with identical names do not overwrite existing files on the server.
- **Filename Sanitization:** The script cleans spaces and special characters from the original file name using regular expressions.
- **File Size Limitation:** The system rejects any file larger than 10MB to protect server storage limits and bandwidth.

9.4 Feature Screenshots & Descriptions

Screenshot 9.1: Administrator Media Upload Form (admin/upload_media.php**)**



Description: Shows the administrator interface used to upload promotional videos and event photographs, link them to specific campus activities, and save them to the database.

10. Error Handling and Validation

10.1 Overview of Validation Strategy

As required by the assessment guidelines in a reliable web application must implement robust validation and error handling mechanisms. The CampusHub system uses a two-layered approach to validate user input and manage errors securely:

1. **Client-Side Validation (Browser Level):** This gives immediate feedback to the student before the form is even submitted, which saves time and reduces unnecessary processing on the server.

2. **Server-Side Validation (PHP Backend Level):** This acts as the primary security defense, ensuring data is clean and correct even if the client-side checks are bypassed or disabled by a user.

10.2 Client-Side Validation Implementation
Client-side validation is handled using native HTML5 attributes. This fulfills the requirement to implement validation efficiently without needing heavy external JavaScript libraries.

- **Mandatory Fields:** The required attribute stops users from submitting empty forms.
- **Email Format:** Using type="email" ensures the student enters a valid email syntax (like student@campus.edu).
- **Phone Numbers:** Using type="tel" helps prevent users from entering invalid text characters in phone number fields.
- **Numeric Limits:** Attributes like min="1" and max="500" on the event seat capacity inputs stop administrators from entering negative or unrealistic numbers.
- **Accepted File Formats:** The accept="image/*,video/*" attribute restricts the file upload dialog so users can only select valid media formats.

HTML

```
<!-- Example of Validated Client Form -->
<input type="email" id="email" name="email" placeholder="student@campus.edu" required>
<input type="password" id="password" name="password" minlength="6" required>
<input type="number" id="seats" name="seats" min="1" max="500" required>
```

10.3 Server-Side Validation & Sanitization
Every piece of data submitted through $_POST and $_GET methods is carefully sanitized and verified by the backend PHP scripts before it is sent to the database. This directly addresses the requirement to manage errors and validate data on the server side.

The custom clean_input() function trims extra spaces, removes slashes, and converts special characters to HTML entities to prevent malicious code injection.

PHP

```
<?php
// Example: Validating User Registration Input securely
$errors = [];

$first_name = clean_input($_POST['first_name'] ?? '');
$last_name  = clean_input($_POST['last_name'] ?? '');
// Validating email format using built-in PHP filters
$email      = filter_var(trim($_POST['email'] ?? ''), FILTER_VALIDATE_EMAIL);
$phone      = trim($_POST['phone'] ?? '');
$password   = $_POST['password'] ?? '';
```

```
// Check required fields and gather errors
if (empty($first_name)) {
   $errors[] = "First name is required.";
}
if (!$email) {
   $errors[] = "Please provide a valid email address.";
}
if (strlen($password) < 6) {
   $errors[] = "Password must be at least 6 characters long.";
}

// Database check for duplicate phone numbers
if (empty($errors)) {
   $check = Database::search("SELECT user_id FROM users WHERE phone = ?", "s", [$phone]);
   if ($check && $check->num_rows > 0) {
      $errors[] = "A user with this phone number already exists.";
   }
}
?>
```

10.4 Error Handling & User Feedback

Proper error management is critical for a smooth user experience and secure system architecture. The system handles errors in three main ways:

1. Visual Alert Messages

When a validation rule fails, the system stops the process and shows a clear, styled alert box to the user so they know exactly what to fix.

PHP

```
<?php if (!empty($error_msg)): ?>
  <div class="alert alert-danger" style="background: #FEE2E2; color: #991B1B; padding: 12px; border-radius: 8px;">
    <strong>Error:</strong> <?php echo htmlspecialchars($error_msg); ?>
  </div>
<?php elseif (!empty($success_msg)): ?>
  <div class="alert alert-success" style="background: #D1FAE5; color: #065F46; padding: 12px; border-radius: 8px;">
    <strong>Success:</strong> <?php echo htmlspecialchars($success_msg); ?>
  </div>
<?php endif; ?>
```

2. Database Error Handling

To keep the application secure, database connection issues are caught gracefully. Using mysqli_report(MYSQLI_REPORT_OFF) ensures that sensitive database error messages (like table names or passwords) are never shown to the end user. If a connection fails, the system logs the error privately and safely stops the script.

3. Session & Route Guard Protection

Administrative pages check the user's role at the very top of the script. If a regular student or unauthenticated visitor tries to access an admin page, the system catches the error and immediately redirects them back to the login page to protect the application.

11. Conclusion

11.1 Project Summary

The **CampusHub** web application was successfully designed, developed, and tested to fulfill all requirements specified in the Web Programming I assignment scenario.

The platform replaces manual tracking, disconnected spreadsheets, and informal communication channels with a centralized, automated web solution. It provides registered students with an easy-to-use interface to browse campus events, book seats online, manage their profiles, view urgent notices, and upload activity media. Concurrently, it equips administrators with a secure dashboard to manage student records, publish announcements, maintain event schedules, upload media assets, and export structured event data.

11.2 Key Learning Outcomes & Technical Accomplishments

Developing CampusHub provided comprehensive, hands-on experience across core web development technologies:

- **Semantic HTML5 & Responsive CSS3:** Implemented clean semantic markup and custom CSS stylesheets utilizing CSS Grid, Flexbox, and media queries to ensure full responsiveness across mobile, tablet, and desktop devices.
- **Server-Side PHP Scripting:** Implemented procedural backend logic, session-based authentication, modular includes (require_once), custom utility functions, and form request handling using $_GET and $_POST methods.
- **Database Connectivity & CRUD Operations:** Designed a normalized MySQL database schema and executed secure Create, Read, Update, and Delete queries via PHP's mysqli extension using prepared statements to prevent SQL injection.
- **XML Data Handling:** Generated well-formed, valid XML documents using PHP's SimpleXMLElement extension to support event data exports and external reporting.
- **Multimedia Integration & File Uploads:** Built a secure file upload pipeline featuring file type whitelisting, unique filename generation, and size validation, alongside native HTML5 audio and video playback components.
- **Defensive Programming & Validation:** Implemented a two-tier validation system combining client-side HTML5 attributes with server-side sanitization (htmlspecialchars, filter_var) to prevent cross-site scripting (XSS) and maintain data integrity.

11.3 Potential Future Enhancements

While the current implementation fulfills all functional and academic objectives, several enhancements could be introduced in future iterations:

- **QR Code Attendance Verification:** Generating unique QR codes upon event registration that organizers can scan at venue entry points.
- **Payment Gateway Integration:** Incorporating online payment processor gateways to handle paid student workshops and event ticket sales.

- **Real-Time Notifications:** Implementing WebSocket or push notification services to instantly alert students when urgent announcements are posted.

- **Dedicated Mobile Application:** Developing a companion mobile application that interfaces directly with the system database.

Appendices

Appendix A: Source Code Structure & Code Evidence

Project Directory Structure

Below is the complete file and folder structure of the CampusHub web application. This shows how I organized the public pages; admin panel, student portal, and reusable include files.

```
c:\xampp\htdocs\web\
│
├── index.php                  - Main Public Homepage
├── about.php                  - Organization Information
├── events.php                 - Events Listing & Search Filter
├── event_details.php          - Event Detailed View & Registration
├── register_event.php         - Event Registration Handler
├── announcements.php          - Notice Board
├── announcement_details.php   - Single Announcement View
├── gallery.php                - Interactive Photo & Video Gallery
├── contact.php                - Contact Form
├── login.php                  - User Authentication Form
├── loginProccess.php          - Server Login Handler
├── logout.php                 - Session Termination
├── register.php               - New Student Account Creation
├── forgot_password.php        - Password Reset Workflow
│
├── includes/
│   ├── config.php             - Database Credentials
│   ├── db.php                 - MySQL Database Singleton & Prepared Helper
│   ├── functions.php          - Authentication & Sanitization Helpers
│   ├── header.php             - Reusable Semantic Header & Dynamic Nav
│   └── footer.php             - Reusable Semantic Footer & Dynamic Date
│
├── admin/
│   ├── dashboard.php          - Admin Analytics & Action Hub
│   ├── manage_events.php      - Event List & Management
│   ├── add_event.php          - New Event Form with Banner Upload
│   ├── edit_event.php         - Event Update Interface
│   ├── delete_event.php       - Event Deletion Handler
│   ├── manage_students.php    - Student Records Table
│   ├── edit_student.php       - Student Account Editor
│   ├── manage_registrations.php - Event Attendance Roster
│   ├── manage_announcements.php - Notice Board CRUD
│   ├── upload_media.php       - Gallery Media Uploader
│   ├── export_events.php      - XML Event Data Generator
```

```
│   ├── manage_roles.php          - User Role Manager
│   └── organization_info.php     - System Contact Config
│
├── student/
│   ├── dashboard.php             - Student Portal Dashboard
│   ├── profile.php               - Student Profile & Avatar Upload
│   ├── my_registrations.php      - Registered Events Roster
│   ├── upload_photo.php          - Student Photo Submission
│   └── apply_role.php            - Club Leadership Application
│
├── database/
│   └── campushub.sql             - Complete MySQL Database Schema Script
│
└── assets/
    ├── css/style.css             - Core Stylesheet
    ├── images/                   - Uploaded Media Assets & Banners
    └── js/                       - Client Interaction Scripts
```

Source Code Screenshots

Database Connection & CRUD Helper (includes/db.php**)**

```php
<?php
include_once __DIR__ . '/config.php';

class Database
{
  private static $connection = null;

  public static function getConnection()
  {
    if (self::$connection === null) {
      // Turn off default error reporting to prevent fatal connection crashes
      mysqli_report(MYSQLI_REPORT_OFF);
      $conn = @new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME, 3306);
      if ($conn->connect_error) {
        return null;
      }
      $conn->set_charset("utf8mb4");
      self::$connection = $conn;
    }
    return self::$connection;
  }

  public static function iud($query, $types = "", $params = [])
  {
    $conn = self::getConnection();
    if (!$conn) {
      return false;
    }
    $statement = $conn->prepare($query);
    if (!$statement) {
      return false;
    }
    if (!empty($types) && !empty($params)) {
```

```
      $statement->bind_param($types, ...$params);
    }
    $result = $statement->execute();
    $statement->close();
    return $result;
  }

  public static function search($query, $types = null, $params = [])
  {
    $conn = self::getConnection();
    if (!$conn) {
      return false;
    }
    $statement = $conn->prepare($query);
    if (!$statement) {
      return false;
    }
    if ($types != null && !empty($params)) {
      $statement->bind_param($types, ...$params);
    }
    $statement->execute();
    $result = $statement->get_result();
    $statement->close();
    return $result;
  }
}
```

Description: Shows the database connection and the parameterized prepared statements used to prevent SQL injection.

File Upload Logic (admin/upload_media.php**)**

```
<?php
require_once '../includes/header.php';
require_once '../includes/db.php';

// Check role permission: super_admin, admin, or moderator
$curr_role = $_SESSION['role'] ?? 'guest';
$curr_user_id = $_SESSION['user_id'] ?? 1;
if ($curr_role !== 'super_admin' && $curr_role !== 'admin' && $curr_role !==
'moderator') {
    echo "<script>window.location.href =
'../student/dashboard.php';</script>";
    exit;
}

$success_msg = "";
$error_msg = "";

// Fetch events
$events = [];
$e_res = Database::search("SELECT event_id, title FROM events ORDER BY
event_date DESC");
if ($e_res && $e_res->num_rows > 0) {
    while ($row = $e_res->fetch_assoc()) {
```

```
        $events[] = $row;
    }
}

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $title = isset($_POST['title']) ? trim($_POST['title']) : '';
    $description = isset($_POST['description']) ? trim($_POST['description'])
: '';
    $raw_type = isset($_POST['media_type']) ? trim($_POST['media_type']) :
'image';
    $media_type = ($raw_type === 'video') ? 'video' : 'image';
    $event_id = isset($_POST['event_id']) ? (int)$_POST['event_id'] : 0;

    // Ensure valid event_id for foreign key constraint
    if ($event_id <= 0 && !empty($events)) {
        $event_id = (int)$events[0]['event_id'];
    }

    $file_path = "hero1.jpg";
    if (isset($_FILES['media_file']) && $_FILES['media_file']['error'] ===
UPLOAD_ERR_OK) {
        $file_name = time() . '_' . basename($_FILES['media_file']['name']);
        $target_dir = __DIR__ . "/../assets/images/";
        $target_file = $target_dir . $file_name;
        if (move_uploaded_file($_FILES['media_file']['tmp_name'],
$target_file)) {
            $file_path = $file_name;
        }
    }

    $meta_json = json_encode(['description' => $description]);

    if (empty($title)) {
        $error_msg = "Media title is required.";
    } elseif ($event_id <= 0) {
        $error_msg = "An active event is required to link media. Please
create an event first.";
    } else {
        $inserted = Database::iud(
            "INSERT INTO media (title, media_type, meta_data,
events_event_id, users_user_id, file_path) VALUES (?, ?, ?, ?, ?, ?)",
            "sssiis",
            [$title, $media_type, $meta_json, $event_id, $curr_user_id,
$file_path]
        );

        if ($inserted) {
            $success_msg = "Media resource published successfully!";
        } else {
            $error_msg = "Could not save media record.";
        }
    }
}
?>
<main class="page-container">
  <style>
    .page-container {
```

```
      --ink: #171b2a;
      --muted: #687083;
      --line: #e4e7ed;
      --coral: #FF6B4A;
      width: 100%;
      padding: 0 24px 64px;
      box-sizing: border-box;
    }

    .form-container {
      width: 100%;
      max-width: 680px;
      margin: 36px auto 0;
    }

    .form-card {
      background: #ffffff;
      border: 1px solid var(--line);
      border-radius: 16px;
      padding: 36px;
      box-shadow: 0 10px 30px rgba(23, 27, 42, 0.08);
    }

    .form-card h1 {
      margin: 0 0 8px;
      color: var(--ink);
      font-size: 1.6rem;
    }

    .form-card p.subtitle {
      margin: 0 0 24px;
      color: var(--muted);
      font-family: 'Cabin Sketch', cursive;
      font-size: 1.15rem;
    }

    .form-group {
      margin-bottom: 20px;
    }

    .form-group label {
      display: block;
      margin-bottom: 8px;
      font-weight: 700;
      color: var(--ink);
      font-size: 0.9rem;
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto,
sans-serif;
    }

    .form-control {
      width: 100%;
      padding: 12px 14px;
      border: 1.5px solid var(--line);
      border-radius: 8px;
      font-size: 0.95rem;
      color: var(--ink);
```

```
      box-sizing: border-box;
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto,
sans-serif;
    }

    .form-control:focus {
      outline: none;
      border-color: var(--coral);
    }

    .btn-submit {
      width: 100%;
      background-color: var(--coral);
      color: #ffffff;
      padding: 14px;
      border: none;
      border-radius: 8px;
      font-weight: bold;
      font-size: 1rem;
      cursor: pointer;
    }

    .alert {
      padding: 14px 18px;
      border-radius: 8px;
      margin-bottom: 20px;
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto,
sans-serif;
      font-size: 0.92rem;
      font-weight: 600;
    }

    .alert-success {
      background-color: #e6f7ec;
      color: #1b7a3e;
      border: 1px solid #c2ebd0;
    }

    .alert-error {
      background-color: #ffeaea;
      color: #c92a2a;
      border: 1px solid #fcc2c2;
    }
  </style>

  <div class="form-container">
    <div class="form-card">
      <h1>Upload Campus Media Asset</h1>
      <p class="subtitle">Publish event banners, promotional flyers, or
gallery albums.</p>

      <?php if (!empty($success_msg)): ?>
        <div class="alert alert-success"><?php echo
htmlspecialchars($success_msg); ?></div>
      <?php endif; ?>

      <?php if (!empty($error_msg)): ?>
```

```
        <div class="alert alert-error"><?php echo
htmlspecialchars($error_msg); ?></div>
      <?php endif; ?>

      <form action="upload_media.php" method="POST" enctype="multipart/form-
data">
        <div class="form-group">
          <label for="title">Title *</label>
          <input type="text" id="title" name="title" class="form-control"
placeholder="e.g. Hackathon Official Poster" required>
        </div>

        <div class="form-group">
          <label for="media_type">Media Type</label>
          <select id="media_type" name="media_type" class="form-control">
            <option value="photo">Photo / Image</option>
            <option value="video">Video Recording</option>
            <option value="audio">Audio Podcast</option>
          </select>
        </div>

        <div class="form-group">
          <label for="event_id">Link to Event</label>
          <select id="event_id" name="event_id" class="form-control">
            <option value="0">General Campus Asset</option>
            <?php foreach ($events as $ev): ?>
              <option value="<?php echo htmlspecialchars($ev['event_id']);
?>">
                <?php echo htmlspecialchars($ev['title']); ?>
              </option>
            <?php endforeach; ?>
          </select>
        </div>

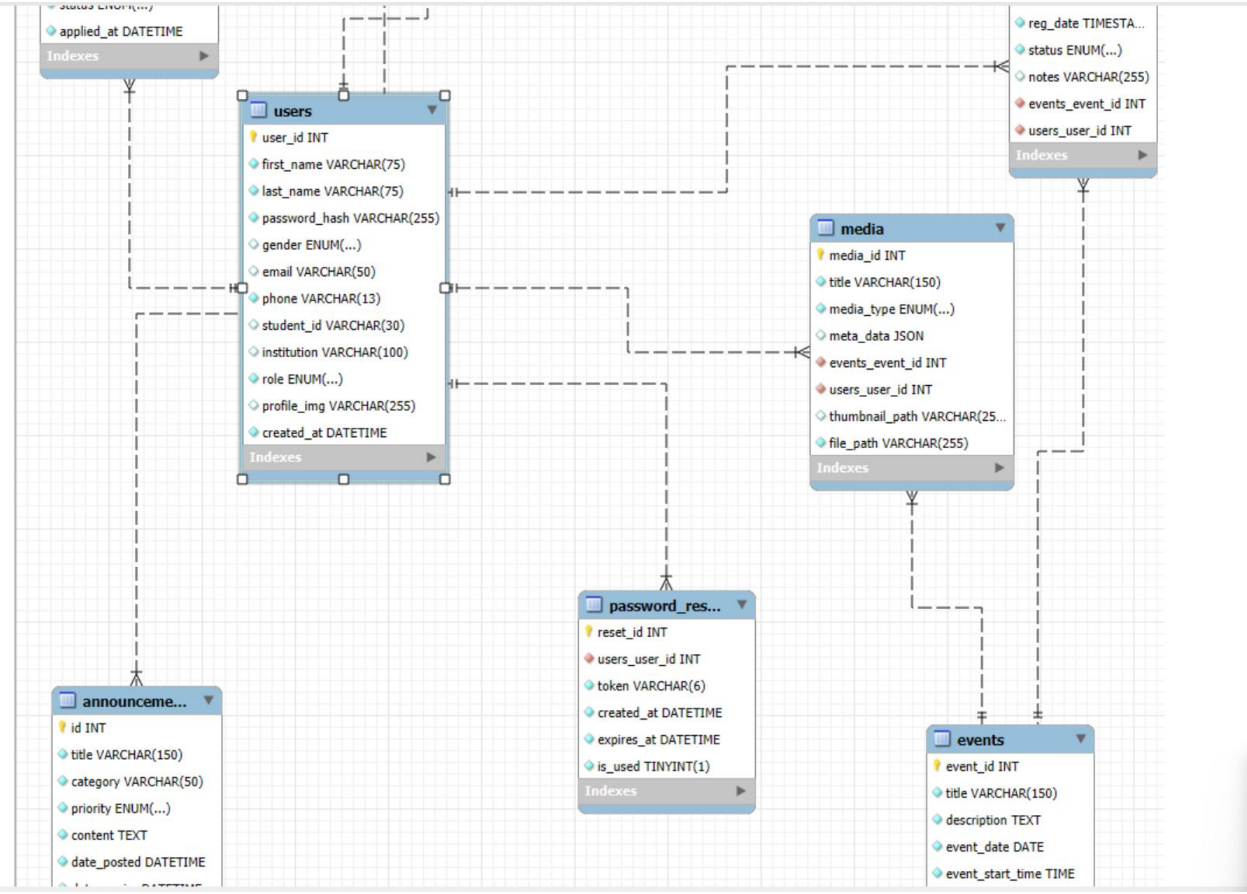
        <div class="form-group">
          <label for="description">Description</label>
          <textarea id="description" name="description" class="form-control"
rows="3" placeholder="Brief note about this asset..."></textarea>
        </div>

        <div class="form-group">
          <label for="media_file">Select File</label>
          <input type="file" id="media_file" name="media_file" class="form-
control">
        </div>

        <button type="submit" class="btn-submit">Upload Asset</button>
      </form>
    </div>
  </div>
</main>
<?php
require_once '../includes/footer.php';
?>
```

Description: Shows the validation code for checking file sizes and extensions before uploading media to the server.

Database Structure Screenshots



Database Table Inventory (campushub **database)**

The following tables were created to store the application's data safely and efficiently.

Table Name	Description	Primary Key	Foreign Keys
users	Stores student and staff accounts	user_id	None
admin	Assigns admin roles (moderator, admin)	admin_id	users_user_id
events	Stores event dates, capacities, and banners	event_id	None
registrations	Stores student event registration bookings	registration_id	events_event_id, users_user_id

Table Name	Description	Primary Key	Foreign Keys
media	Stores uploaded photos and videos	media_id	events_event_id, users_user_id
announcements	Stores campus notices and priority tags	id	admin_admin_id
master_control	Stores organizational contact email	id	None

Database Screenshots

Name	Auto Increme...	Modified Date	Data Length	Engine	Rows	Comment
admin	4	2026-08-27 14:16:13	16 KB	InnoDB	3	
announcements	2	2026-08-26 16:39:38	16 KB	InnoDB	1	
events	3	2026-08-27 16:50:48	16 KB	InnoDB	2	
master_control	1		16 KB	InnoDB	0	
media	2	2026-08-28 12:43:46	16 KB	InnoDB	1	
password_resets	1		16 KB	InnoDB	0	
registrations	2		16 KB	InnoDB	1	
role_applications	1		16 KB	InnoDB	0	
users	8	2026-08-27 14:03:34	16 KB	InnoDB	4	

admin_id # int	users_user_id # int	role enum
1	1	super_adm
2	3	moderator
3	7	admin

Table Profile | Begin Transaction | Cell Editor | Filter & Sort | Columns | Data Profiling | Tools

id # int	title varchar(150)	category varchar(50)	priority enum	content text	date_posted datetime	date_expire datetime	admin_admin_id # int	meta_data json	status enum
1	test	General	medium	test	2026-08-26 10:1	2026-08-29 10:	1	{"name": "test"	display

Objects | events @campushub (localhost_3306) -...

Table Profile | Begin Transaction | Cell Editor | Filter & Sort | Columns | Data Profiling | Tools

event_id # int	title varchar(150)	description text	event_date date	event_start_time time	event_end_time time	img_path varchar(255)	meta_data json	status enum	seats # int
1	test	test	2026-08-27	16:40:37	22:40:40	assets/images/ev	(Null)	display	100
2	Cricket match	test two	2026-09-08	09:00:00	18:30:00	assets/images/ev	{"tags": ["#cricl	display	150

Objects | media @campushub (localhost_3306) -...

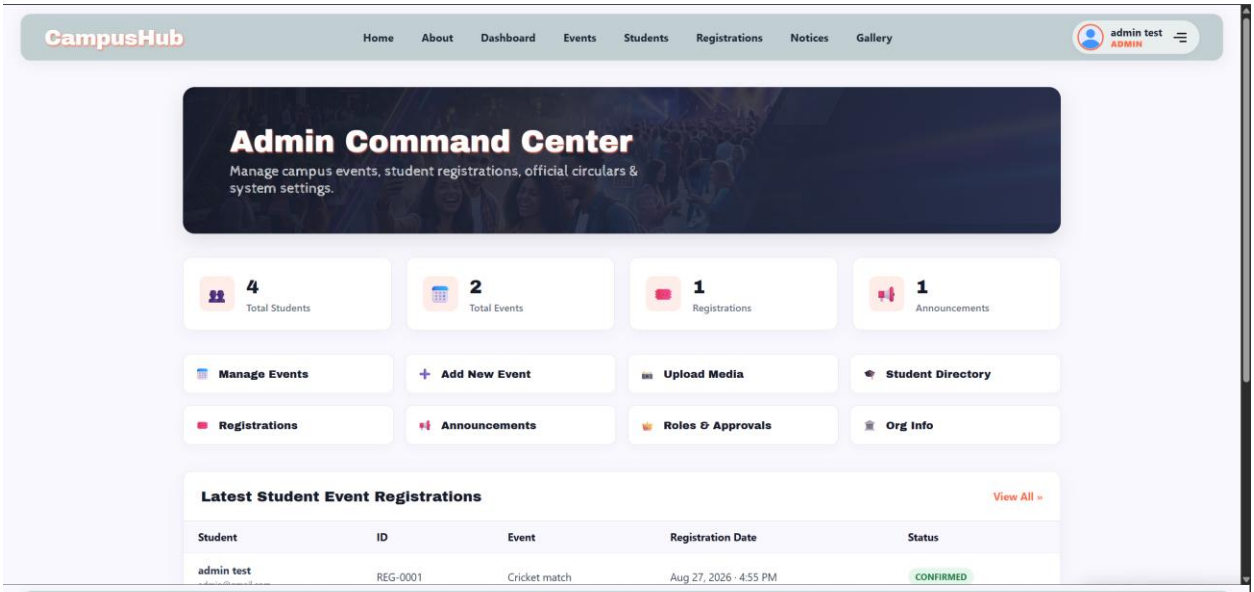
Table Profile | Begin Transaction | Cell Editor | Filter & Sort | Columns | Data Profiling | Tools

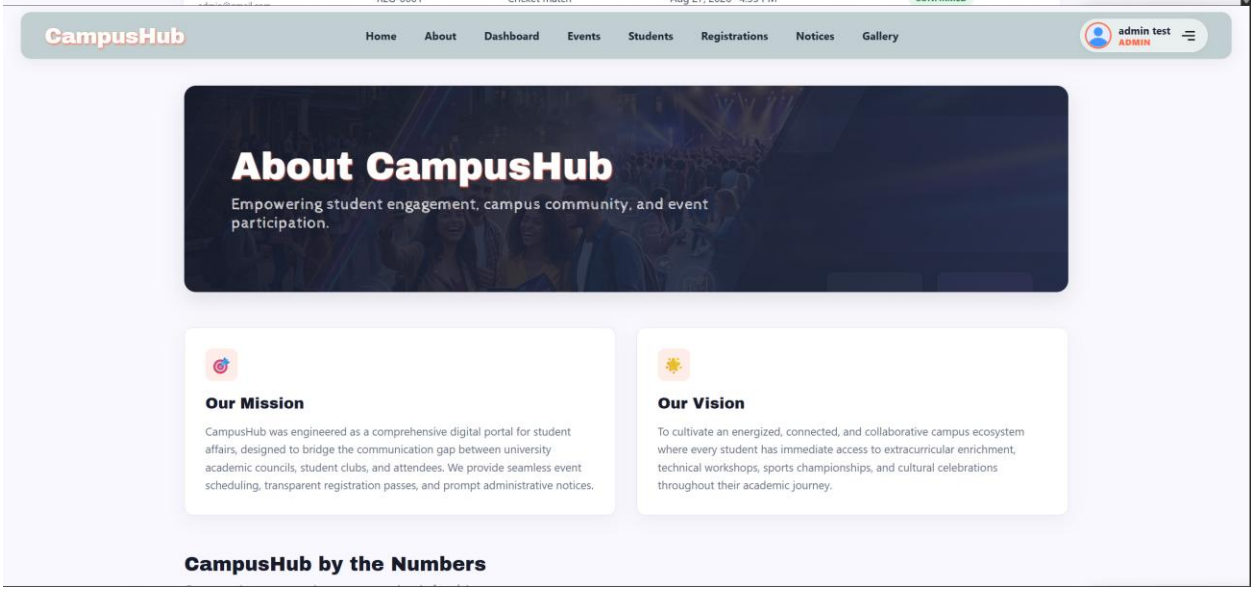
media_id # int	title varchar(150)	media_type enum	meta_data json	events_event_id # int	users_user_id # int	thumbnail_path varchar(255)	file_path varchar(255)
1	test	image	{"description": "test"}	2	1	(Null)	1787901226_Generated Ir

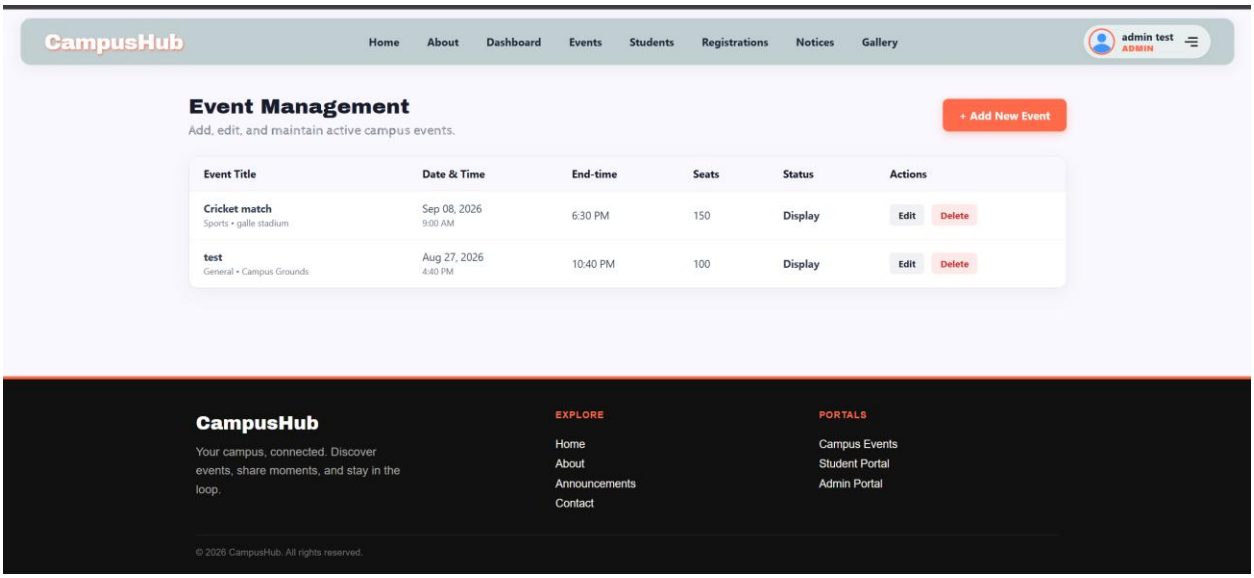
Objects | users @campushub (localhost_3306) - ...

Table Profile | Begin Transaction | Cell Editor | Filter & Sort | Columns | Data Profiling | Tools

user_id # int	first_name varchar(75)	last_name varchar(75)	password_hash varchar(255)	gender enum	email varchar(50)	phone varchar(13)	student_id varchar(30)	institution varchar(100)	role enum	profile_img varchar(255)	created_at datetime
1	test	test	$2y$10$RsDvJt3VsMiKTbI	male	test@gmail.com	+941234567890	(Null)	CampusHub	student	(Null)	2026-08-28 13:51:18
2	BANUJA	KAUSHALYA	$2y$10$mw1LbmNP/st69	male	btv731563@gmail.c	+94767868297	(Null)	CampusHub	student	(Null)	2026-08-28 13:51:18
3	moderator	test	$2y$10$1DSl48flzgIJ3/9N	male	modarator@gmail.c	+940112233445	(Null)	CampusHub	student	(Null)	2026-08-28 13:51:18
7	admin	test	$2y$10$BRKg44MR6/81X	female	admin@gmail.com	+940762266991	(Null)	CampusHub	student	(Null)	2026-08-28 13:51:18






CampusHub
Home
About
Dashboard
Events
Students
Registrations
Notices
Gallery
admin test
ADMIN
Event Management
Add, edit, and maintain active campus events.
+ Add New Event
Event Title
Date & Time
End-time
Seats
Status
Actions
Cricket match
Sports • galle stadium
Sep 08, 2026
9:00 AM
6:30 PM
150
Display
Edit
Delete
test
General • Campus Grounds
Aug 27, 2026
4:40 PM
10:40 PM
100
Display
Edit
Delete
CampusHub
Your campus, connected. Discover events, share moments, and stay in the loop.
EXPLORE
Home
About
Announcements
Contact
PORTALS
Campus Events
Student Portal
Admin Portal
© 2026 CampusHub. All rights reserved.